

PROYECTO FINAL

Desarrollar una **aplicación de consola en .NET C#** que integre cuatro paradigmas de programación, partiendo del template **ConsoleApp** sin código adicional generado.

Pueden reutilizar y extender el dominio del Taller 1.

PARADIGMA ORIENTADO A OBJETOS

El sistema debe demostrar **POO completa**:

- **Mínimo 5 clases** con relaciones: herencia, composición, agregación y asociación
- **Al menos 1 interfaz** implementada
- **Polimorfismo** demostrable en tiempo de ejecución
- **Diagrama de clases UML actualizado**

PARADIGMA DE ASPECTOS

Funciones transversales con Castle DynamicProxy:

- Configurar **Castle Windsor** como contenedor de Inyección de Dependencias
- Los servicios deben resolverse a través de **interfaces**
- Implementar **mínimo 2 interceptores** para inquietudes transversales:
 - **Logging automático** — registrar entrada/salida de métodos
 - **Validación o manejo de errores** — captura centralizada

PROGRAMACIÓN FUNCIONAL

Aplicar estilo funcional sobre el dominio del sistema:

- **LINQ** para consultas de datos: mínimo Where, Select y Aggregate
- **Funciones puras** sin efectos secundarios
- Uso de **Func<>** / **Action<>** como parámetros de alto orden
- Al menos un **tipo inmutable** (record)

PROGRAMACIÓN ORIENTADA A EVENTOS

Reactividad mediante eventos de C#:

- Definir **mínimo 2 eventos personalizados** con EventArgs propios
- Los eventos deben reflejar **cambios de estado significativos** del dominio

Ejemplo: PedidoCreado, StockActualizado, PagoAprobado

ENTREGABLES

- **Repositorio GitHub público** que contenga:
 - README con descripción del sistema y decisiones de diseño por paradigma
 - Diagrama de clases UML ([Draw.io](https://draw.io))
 - Proyecto .NET C# (con [.gitignore](#) para evitar archivos de build)
 - Commits que reflejen el trabajo de cada integrante
- Demostración en clase de **cada paradigma en funcionamiento**

RECOMENDACIONES

- Elijan un dominio con **comportamiento rico**: pedidos, reservas, inventario, turnos médicos, etc.
- El dominio de POO es la **base** — los otros paradigmas *se aplican sobre él*
- Castle intercepta a través de **interfaces** — no necesitan métodos `virtual`; registren los servicios como su interfaz en el contenedor
- Los eventos deben ser **semánticamente significativos**, no solo técnicos
- Documenten claramente **qué parte del código corresponde a cada paradigma**